

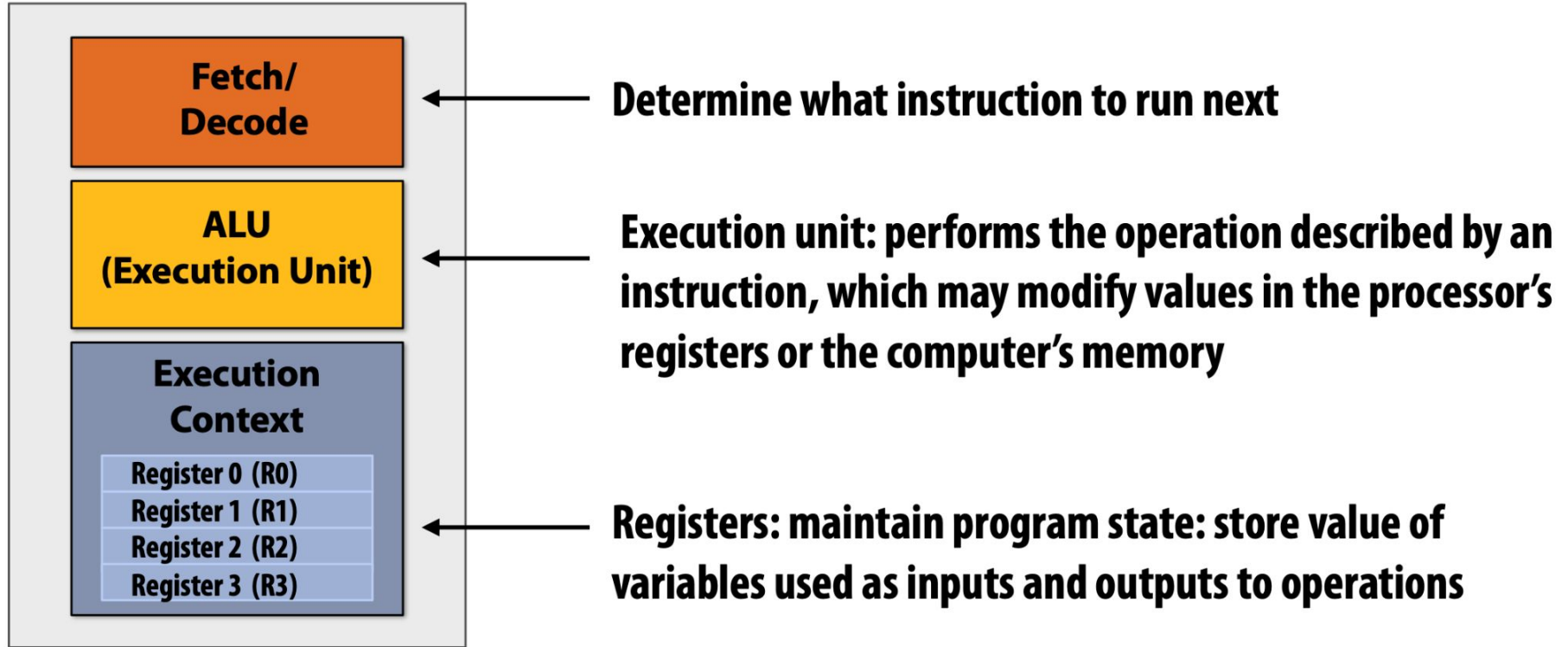
Parallel Computer Architecture

Lecture 3:

- Intuition for Parallelism

Armin Mehrabian

Spring 2025



A Programs Perspective of Memory

A computer's memory is structured as an array of bytes.

- Each byte is assigned a unique **address**, which represents its position in the memory array.

(We'll continue assuming the memory is byte-addressable.)

- **Examples:**

- The byte at address **0x2** contains the value **1**.
- The byte at address **0x1F (31)** contains the value **255**.

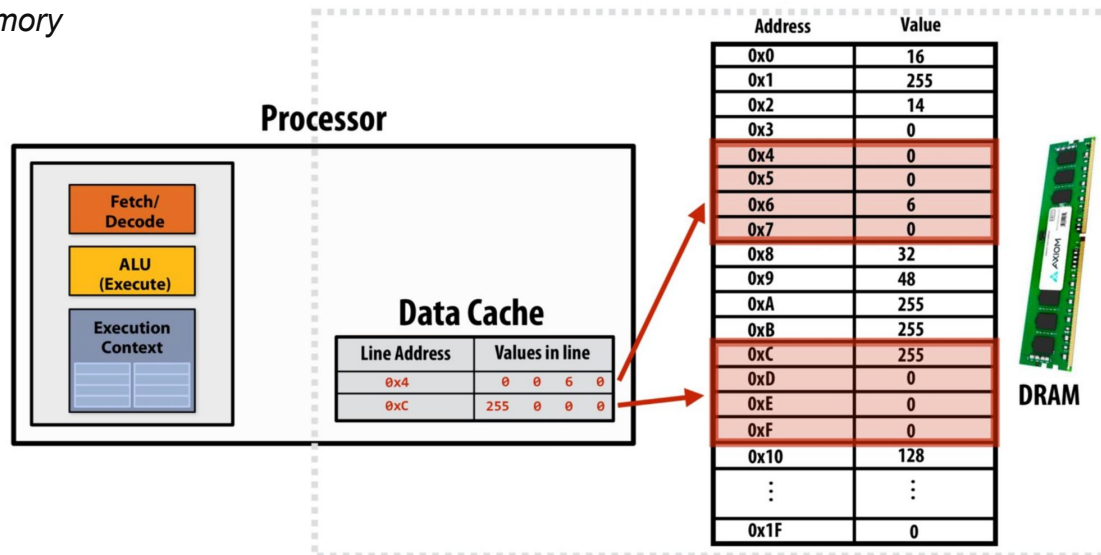
Memory	
Address	Value
0x0	10
0x1	0
0x2	1
...	
0x1F	255

What Are Caches?

A cache is a small, high-speed hardware storage layer that holds frequently accessed data to reduce memory access latency.

Key Properties of Caches

- **Transparent to Software**
 - Does not alter program behavior.
 - Only impacts performance.
- **On-Chip Storage**
 - Stores a subset of data from main memory.
 - Faster access than DRAM (e.g., 1-5 cycles vs. 100+ cycles).
- **Cache Line Granularity**
 - Data stored in fixed-size blocks (*cache lines*).
 - Example: 4-byte line size.
- **Two ways caches help**
 - Spatial and Temporal locality



Cache example 1

Array of 16 bytes in memory

	Address	Value
Line 0x0	0x0	16
	0x1	255
	0x2	14
	0x3	0
Line 0x4	0x4	0
	0x5	0
	0x6	6
	0x7	0
Line 0x8	0x8	32
	0x9	48
	0xA	255
	0xB	255
Line 0xC	0xC	255
	0xD	0
	0xE	0
	0xF	0

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU)
replacement policy

Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load 0x0	0x0 ● ● ● ●
0x1	hit	0x0 ● ● ● ●
0x2	hit	0x0 ● ● ● ●
0x3	hit	0x0 ● ● ● ●
0x2	hit	0x0 ● ● ● ●
0x1	hit	0x0 ● ● ● ●
0x4	"cold miss", load 0x4	0x0 ● ● ● ● 0x4 ● ● ● ●
0x1	hit	0x0 ● ● ● ● 0x4 ● ● ● ●

time ↓

There are two forms of "data locality" in this sequence:

Spatial locality: loading data in a cache line "preloads" the data needed for subsequent accesses to different addresses in the same line, leading to cache hits

Temporal locality: repeated accesses to the same address result in hits.

Cache example 2

Array of 16 bytes in memory

	Address	Value
Line 0x0	0x0	16
	0x1	255
	0x2	14
	0x3	0
Line 0x4	0x4	0
	0x5	0
	0x6	6
	0x7	0
Line 0x8	0x8	32
	0x9	48
	0xA	255
	0xB	255
Line 0xC	0xC	255
	0xD	0
	0xE	0
	0xF	0

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU)
replacement policy

Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load 0x0	0x0 ● ● ● ●
0x1	hit	0x0 ● ● ● ●
0x2	hit	0x0 ● ● ● ●
0x3	hit	0x0 ● ● ● ●
0x4	"cold miss", load 0x4	0x0 ● ● ● ● 0x4 ● ● ● ●
0x5	hit	0x0 ● ● ● ● 0x4 ● ● ● ●
0x6	hit	0x0 ● ● ● ● 0x4 ● ● ● ●
0x7	hit	0x0 ● ● ● ● 0x4 ● ● ● ●
0x8	"cold miss", load 0x8 (evict 0x0)	0x8 ● ● ● ● 0x4 ● ● ● ●
0x9	hit	0x8 ● ● ● ● 0x4 ● ● ● ●
0xA	hit	0x8 ● ● ● ● 0x4 ● ● ● ●
0xB	hit	0x8 ● ● ● ● 0x4 ● ● ● ●
0xC	"cold miss", load 0xC (evict 0x4)	0x8 ● ● ● ● 0xC ● ● ● ●
0xD	hit	0x8 ● ● ● ● 0xC ● ● ● ●
0xE	hit	0x8 ● ● ● ● 0xC ● ● ● ●
0xF	hit	0x8 ● ● ● ● 0xC ● ● ● ●
0x0	"capacity miss", load 0x0 (evict 0x8)	0x0 ● ● ● ● 0xC ● ● ● ●

time

How Does a Processor Decide What Data to Keep in Cache?

Cache Mapping Policies (*Simplified Overview*)

1. **Direct-Mapped Cache**
 - Each memory block maps to **exactly one location** in the cache.
2. **Set-Associative Cache**
 - Each memory block maps to a **set** of cache locations (e.g., 4-way associative).
3. **Fully Associative Cache**
 - Data can be placed **anywhere** in the cache (rarely used).

1. Direct-Mapped Cache

Rule: Each memory address maps to **exactly one cache line**.

Example:

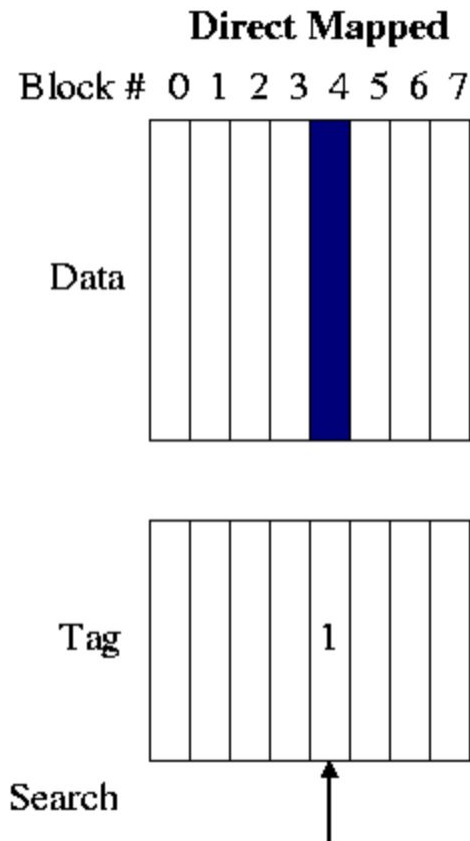
- **Cache Size:** 4 lines (0-3).
- **Memory Addresses:** 0x00, 0x04, 0x08, 0x0C.
- **Mapping Formula:**
$$\text{Cache Line} = (\text{Address} / \text{Block Size}) \% \text{Total Lines}.$$

Pros ✓

- **Simple hardware:** Easy to implement (no complex search logic).
- **Fast access time:** Fixed mapping means no decision-making overhead.
- **Low power consumption:** Minimal circuitry required.

Cons ✗

- **High conflict misses:** Multiple addresses compete for the same cache line.
- **Inflexible replacement:** No choice in eviction—forced replacement on collision.
- **Poor utilization:** Frequently accessed blocks may evict each other.



2. Set-Associative Cache

Rule: Each memory address maps to a **set** (e.g., 2 lines per set).

Example:

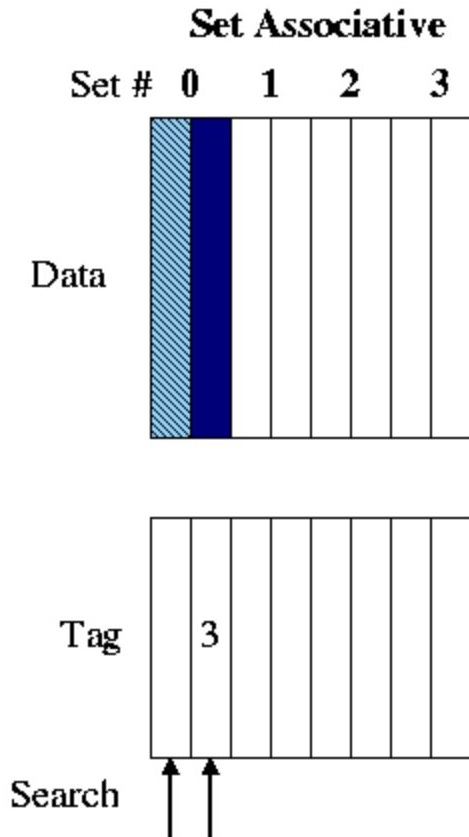
- **Cache:** 4 sets $\times$ 2 lines = 8 total lines.
- **Memory Addresses:** 0x00, 0x08, 0x10.
- **Mapping Formula:** $\text{Set} = (\text{Address} / \text{Block Size}) \% \text{Total Sets}$.

Pros ✓

- **Reduced conflict misses:** Multiple slots per set reduce collisions.
- **Balanced performance:** Better hit rate than direct-mapped, simpler than fully associative.
- **Flexible replacement:** Policies like LRU can be applied *within a set*.

Cons ✗

- **Increased complexity:** Additional logic to manage sets and replacement policies.
- **Slower than direct-mapped:** Slightly longer access time due to set search.
- **Higher power usage:** More circuitry for tag comparison.



3. Fully-Associative Cache

Rule: Any memory address can map to **any** cache line.

Example:

- **Cache Size:** 4 lines.
- **Memory Addresses:** 0x00, 0x04, 0x08, 0x0C, 0x10.

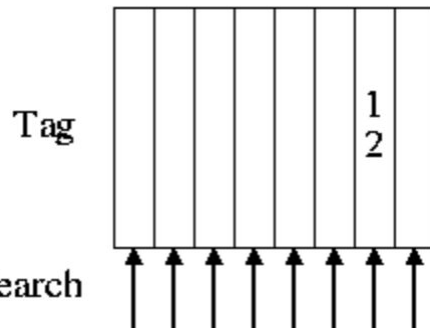
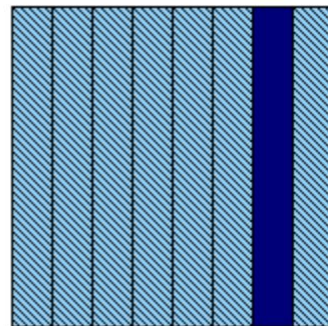
Pros ✓

- **Minimal conflict misses:** Any block can occupy any cache line.
- **Optimal replacement:** Global LRU policy maximizes cache utilization.
- **Flexibility:** Ideal for small, high-priority caches (e.g., TLB).

Cons ✗

- **High complexity:** Requires searching all entries for a match (slow).
- **Impractical for large sizes:** Hardware cost grows exponentially with cache size.
- **Power-intensive:** Parallel tag comparison consumes significant energy.

Fully Associative



Example Program

```
int main() {  
    int result = 1;  
    for (int i = 0; i < 10; i++) {  
        result = result * 2;  
    }  
    printf("%d\n", result);  
    return 0;  
}
```

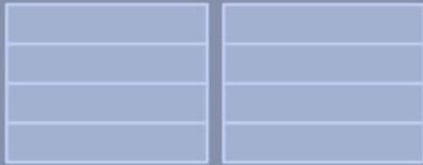
Compile

```
ld    r0, addr[r1]  
_____  
mul   r1, r0, r0  
_____  
mul   r1, r1, r0  
_____  
...  
_____  
st    addr[r2], r0  
v
```

**Fetch/
Decode**

**Execution Unit
(ALU)**

**Execution
Context**



```
ld    r0, addr[r1]
```

```
mul   r1, r0, r0
```

```
mul   r1, r1, r0
```

```
...
```

```
...
```

```
...
```

```
...
```

```
...
```

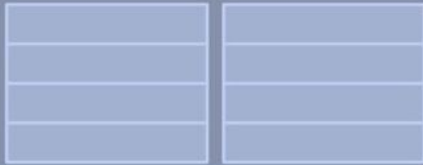
```
...
```

```
st    addr[r2], r0
```

**Fetch/
Decode**

**Execution Unit
(ALU)**

**Execution
Context**



ld r0, addr[r1]

mul r1, r0, r0

mul r1, r1, r0

...

...

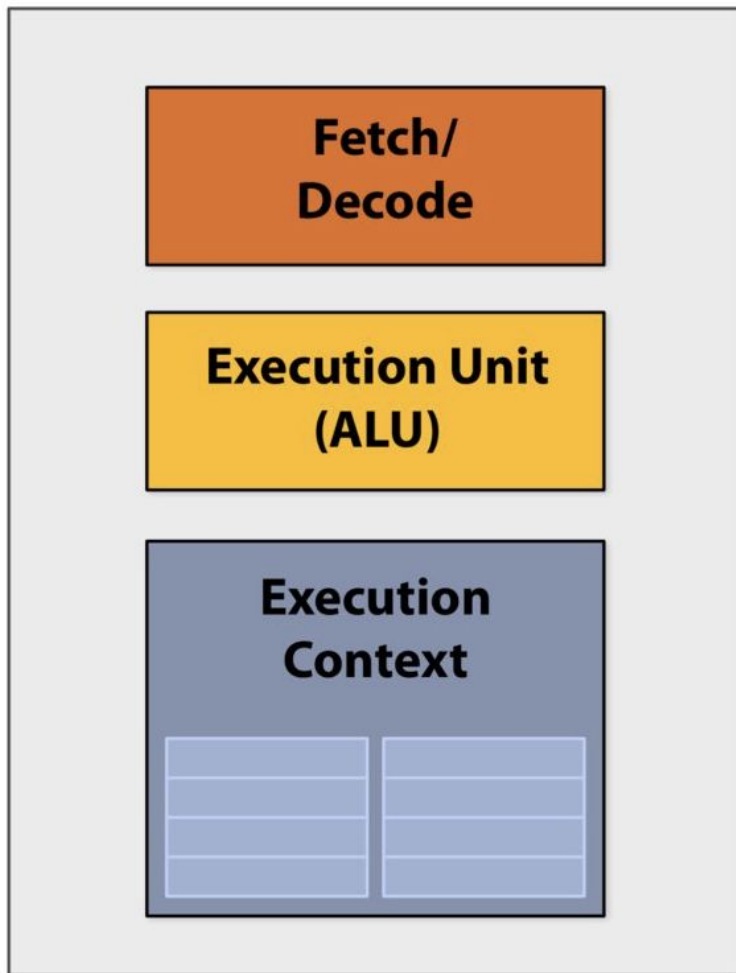
...

...

...

...

st addr[r2], r0



```
ld    r0, addr[r1]
```

```
mul   r1, r0, r0
```

```
mul   r1, r1, r0
```

```
...
```

```
...
```

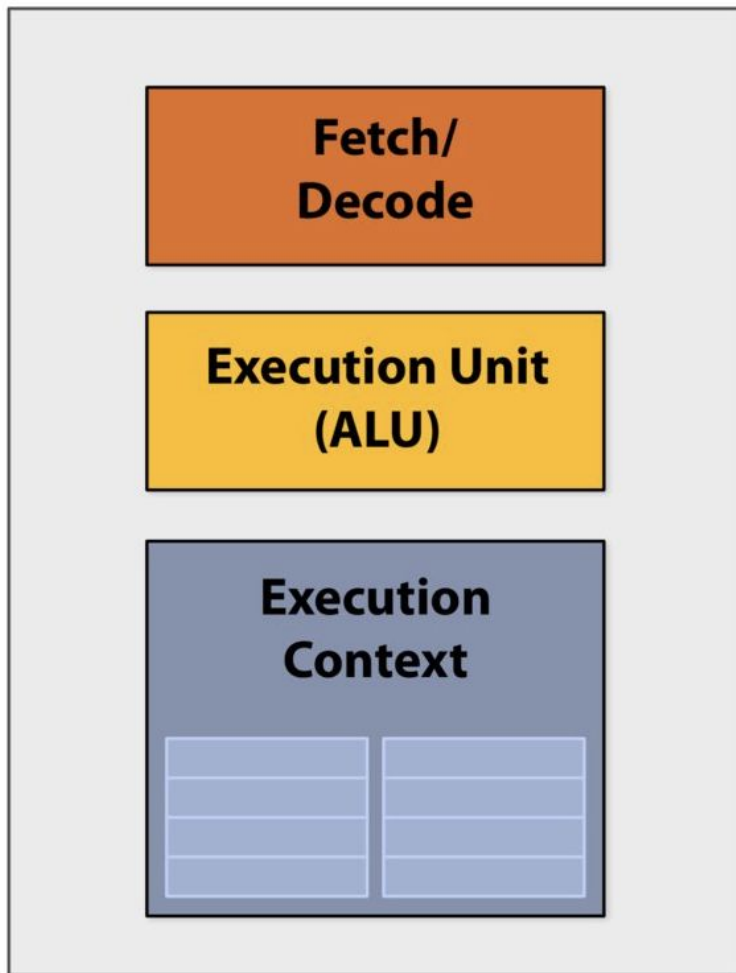
```
...
```

```
...
```

```
...
```

```
...
```

```
st    addr[r2], r0
```



```
ld    r0, addr[r1]
mul   r1, r0, r0
mul   r1, r1, r0
...
...
...
...
...
...
st    addr[r2], r0
```

Instruction-Level Parallelism (ILP)

Executing multiple instructions simultaneously within a single program thread.

Key Techniques

1. Superscalar Execution

- Multiple instructions decoded/executed per clock cycle.
- Example: Intel/AMD processors.

2. Pipelining

- Overlap stages of instruction processing (Fetch, Decode, Execute, Writeback).

3. Out-of-Order Execution

- Dynamically reorder instructions to avoid stalls.

```
1. a = x + y  
2. b = z * 2  
3. c = a + b  
4. d = m - n
```

Independent → Can run in parallel
(Depends on 1 and 2)
→ Independent of 1-3

Example Program with ILP

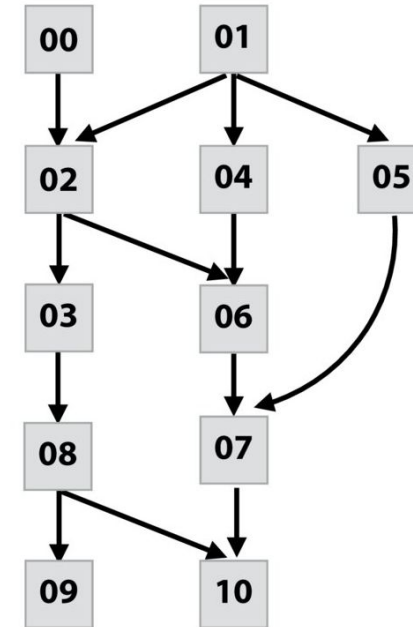
Program (sequence of instructions)

PC	Instruction	
00	a = 2	
01	b = 4	
02	tmp2 = a + b	// 6
03	tmp3 = tmp2 + a	// 8
04	tmp4 = b + b	// 8
05	tmp5 = b * b	// 16
06	tmp6 = tmp2 + tmp4	// 14
07	tmp7 = tmp5 + tmp6	// 30
08	if (tmp3 > 7)	
09	print tmp3	
	else	
10	print tmp7	

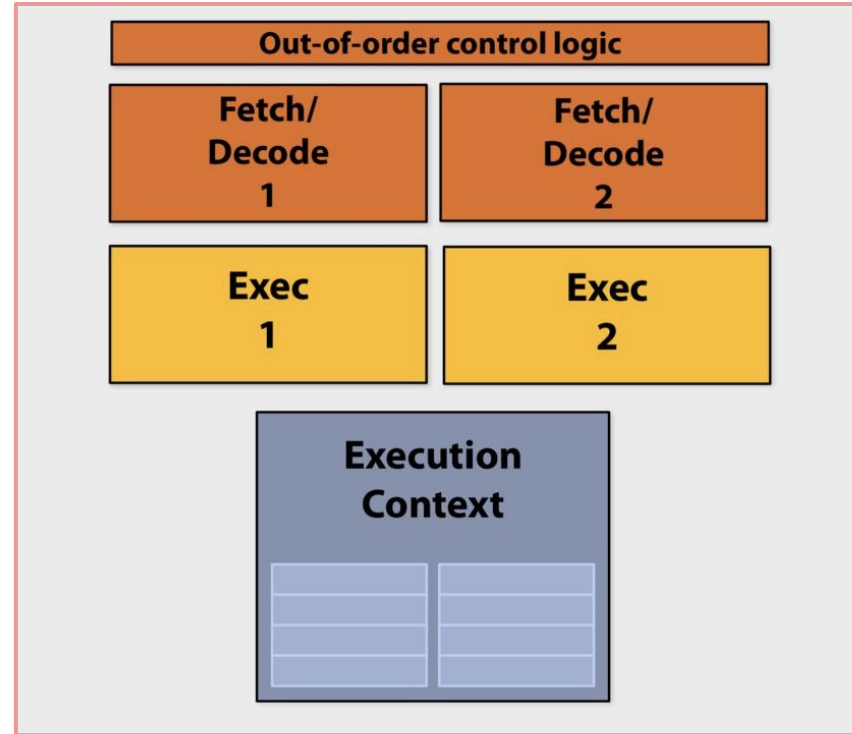
value during execution



Instruction dependency graph



ILP: One processor, Multiple Execution Unit



Example Program

Inputs (**x**):

- A single 1D array containing **both masses and velocities** for the 2D grid:
 - The first half of **x** contains **mass values** (**x[index]**).
 - The second half contains **velocity values** (**x[index + M]**).
- Organized in a row-major format for a grid with dimensions **N x M**.

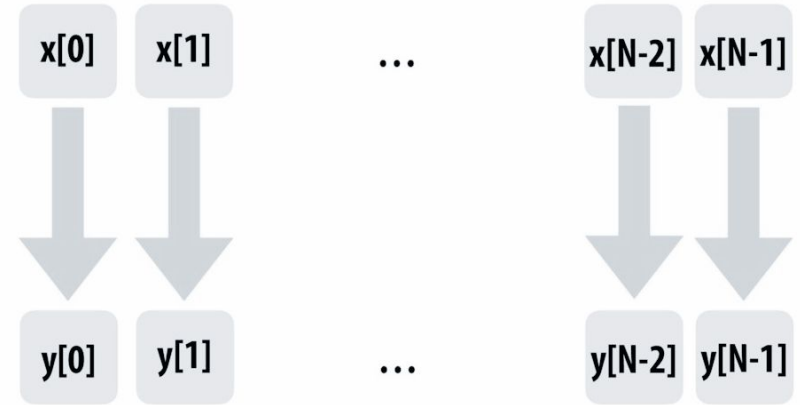
Outputs (**y**):

- A 1D array where each element represents the **total kinetic energy** for a row in the grid.
- **y[i]** corresponds to the total energy for the **i**th row.

```
1 void calculateKineticEnergy(int N, int M, float* x, float* y) {
2     for (int i = 0; i < N; i++) {
3         y[i] = 0; // Initialize energy for the current row
4
5         // Inner Loop: Sum the kinetic energy for each particle in the row
6         for (int j = 0; j < M; j++) {
7             int index = i * M + j; // Convert 2D indices to 1D index
8             float v = x[index + M]; // Velocity is stored after mass in x
9             float m = x[index];     // Mass is stored first in x
10            y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2
11        }
12    }
13 }
```

Example Program

```
1 void calculateKineticEnergy(int N, int M, float* x, float* y) {
2     for (int i = 0; i < N; i++) {
3         y[i] = 0; // Initialize energy for the current row
4
5         // Inner Loop: Sum the kinetic energy for each particle in the row
6         for (int j = 0; j < M; j++) {
7             int index = i * M + j; // Convert 2D indices to 1D index
8             float v = x[index + M]; // Velocity is stored after mass in x
9             float m = x[index];     // Mass is stored first in x
10            y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2
11        }
12    }
13 }
```



Example Program

Compile the program → Sequence of Instructions

```
1 void calculateKineticEnergy(int N, int M, float* x, float* y) {  
2   for (int i = 0; i < N; i++) {  
3     y[i] = 0; // Initialize energy for the current row  
4  
5     // Inner Loop: Sum the kinetic energy for each particle in the row  
6     for (int j = 0; j < M; j++) {  
7       int index = i * M + j; // Convert 2D indices to 1D index  
8       float v = x[index + M]; // Velocity is stored after mass in x  
9       float m = x[index];     // Mass is stored first in x  
10      y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2  
11    }  
12  }  
13 }
```

Compile

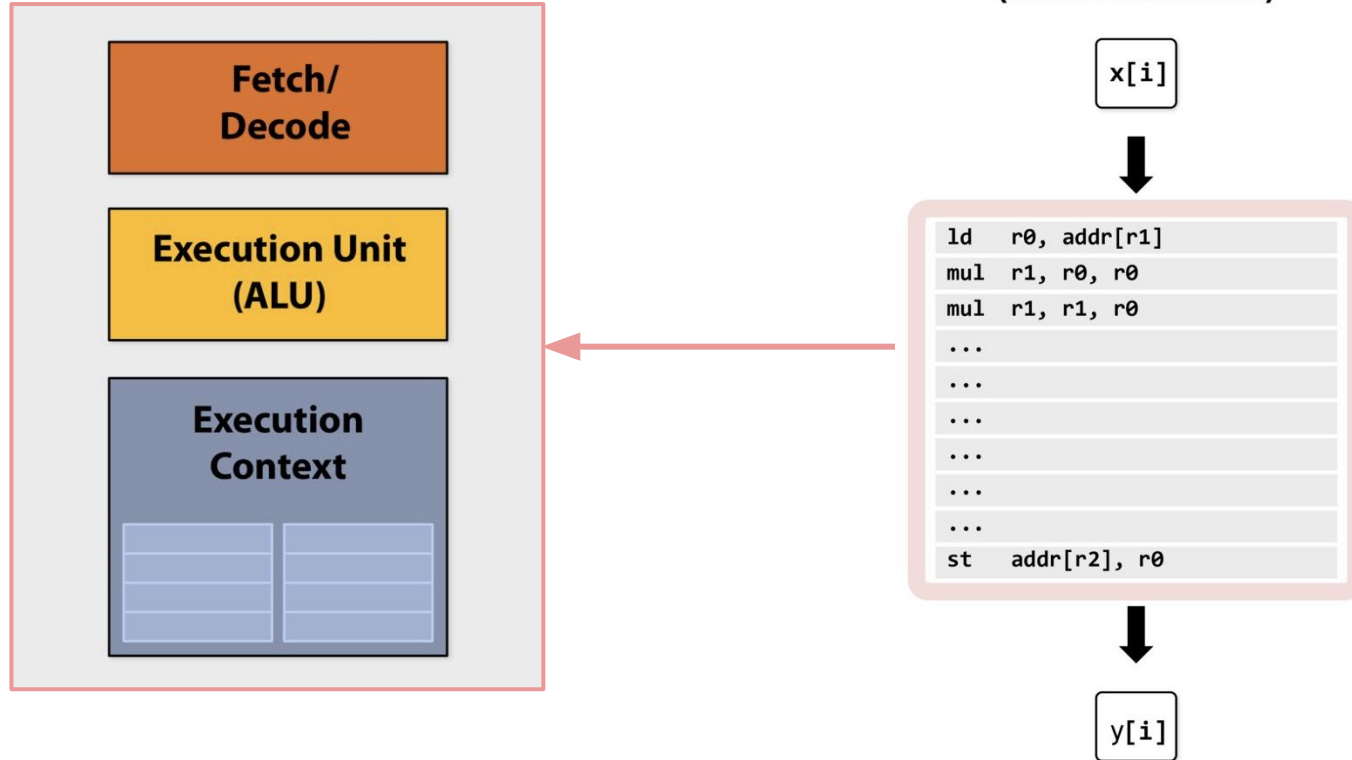
Compiled instruction stream
(scalar instructions)

x[i]

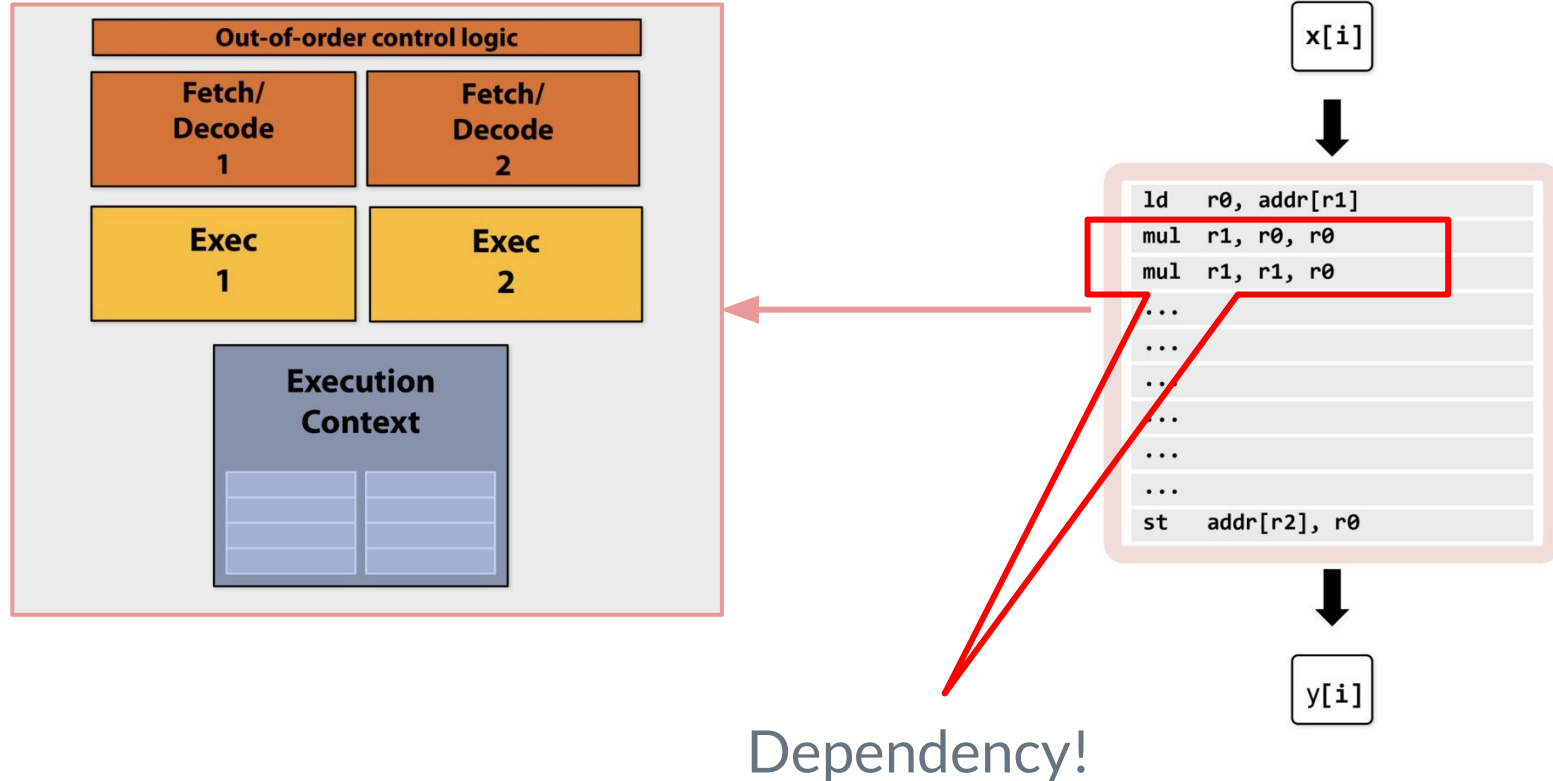
ld	r0, addr[r1]
mul	r1, r0, r0
mul	r1, r1, r0
...	
...	
...	
...	
...	
...	
st	addr[r2], r0

y[i]

Example Program



Example Program: ILP

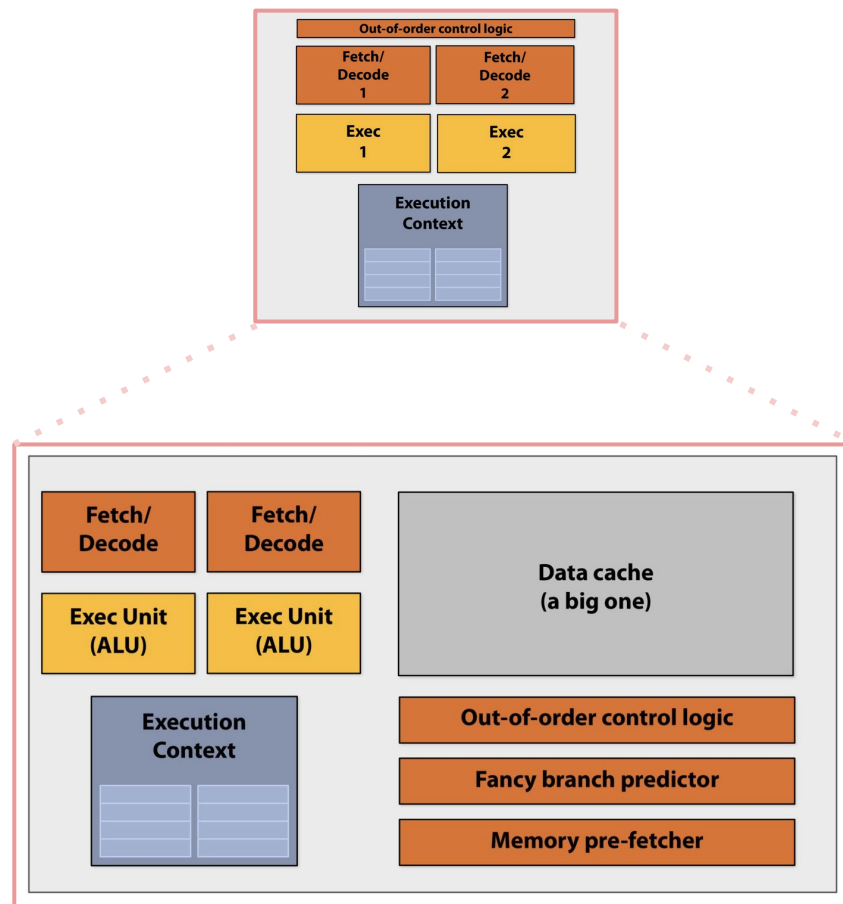


Is there any parallelism left here?

```
1  void calculateKineticEnergy(int N, int M, float* x, float* y) {
2      for (int i = 0; i < N; i++) {
3          y[i] = 0; // Initialize energy for the current row
4
5          // Inner Loop: Sum the kinetic energy for each particle in the row
6          for (int j = 0; j < M; j++) {
7              int index = i * M + j; // Convert 2D indices to 1D index
8              float v = x[index + M]; // Velocity is stored after mass in x
9              float m = x[index];      // Mass is stored first in x
10             y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2
11         }
12     }
13 }
```


ILP Diminishing Return

- The majority of chip transistors are utilized to enhance the speed of executing a single instruction stream.
- **Diminishing Returns:** More transistors for ILP features yield smaller performance gains as ILP approaches its limits.
- **Higher Costs:** Complex ILP logic increases design complexity, power usage, and manufacturing costs.
- **Underutilized Potential:** Many workloads lack sufficient parallelism to fully benefit from advanced ILP optimizations.
- **Better Alternatives:** Transistors could be better spent on more cores or specialized accelerators.
- **Memory Bottlenecks:** Larger caches and smarter predictors can't fully mitigate memory access delays.
- **ILP Saturation:** Techniques like out-of-order execution face hard limits on their effectiveness.



Any other idea for parallelism?

Many-Core vs Single-Core

Old Approach: Enhancing Single-Core Performance

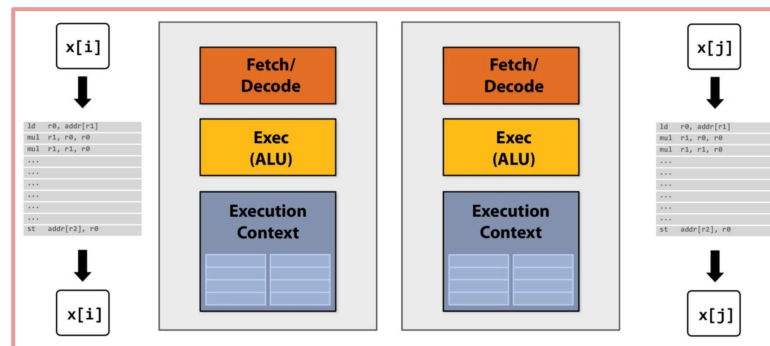
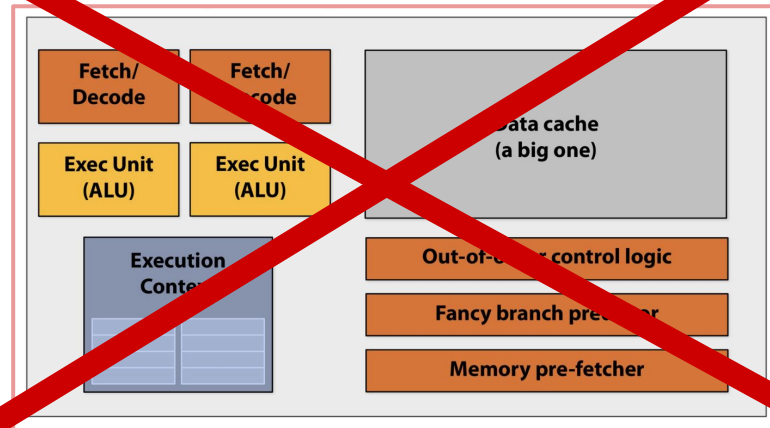
- More transistors → Faster execution of one instruction stream.
- Techniques: Out-of-order execution, speculative operations.

New Approach: Multi-Core Processing

- More transistors → More cores, not just a faster single core.
- Enables parallel execution, better efficiency, and scalability.

Simpler cores:

- each core may be slower at running a single instruction stream than our original "fancy" core
- (e.g., 25% slower) But there are now two cores: $2 \times 0.75 = 1.5$ (potential for speedup!)



But, no parallelism expressed in the code

```
1 void calculateKineticEnergy(int N, int M, float* x, float* y) {  
2     for (int i = 0; i < N; i++) {  
3         y[i] = 0; // Initialize energy for the current row  
4  
5         // Inner Loop: Sum the kinetic energy for each particle in the row  
6         for (int j = 0; j < M; j++) {  
7             int index = i * M + j; // Convert 2D indices to 1D index  
8             float v = x[index + M]; // Velocity is stored after mass in x  
9             float m = x[index];     // Mass is stored first in x  
10            y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2  
11        }  
12    }  
13 }
```

- This C program compiles into a single instruction stream that executes on one thread and one processor core.
- Simple processor cores was 25% slower than the original single complicated one, our program now runs 25% slower than before.

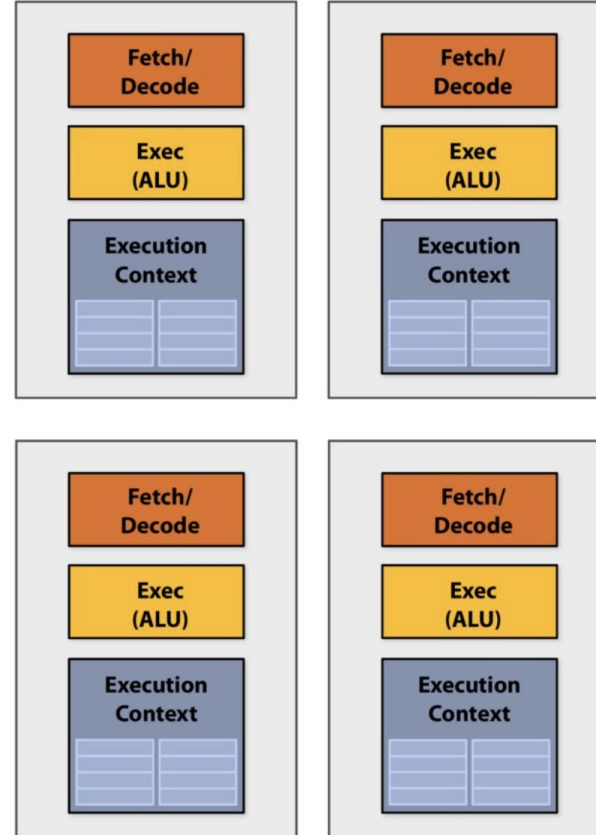
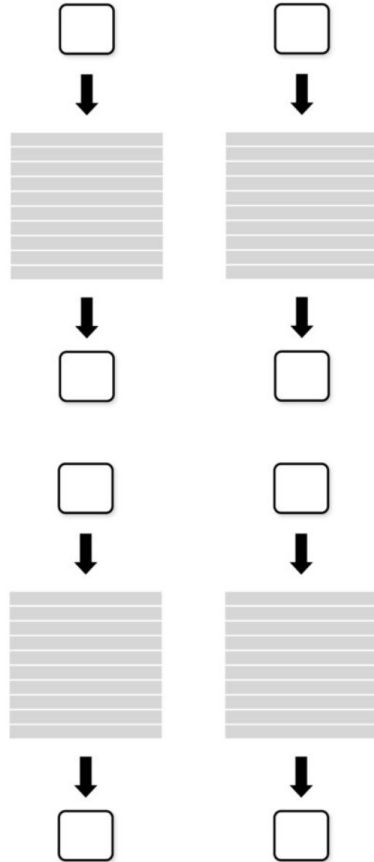
Spawning Multiple Threads

- We can create two threads
- Which generates two instruction stream
- “We” the **programmer**, or sometimes **compiler** has the knowledge that, which parts of the code is *parallel*

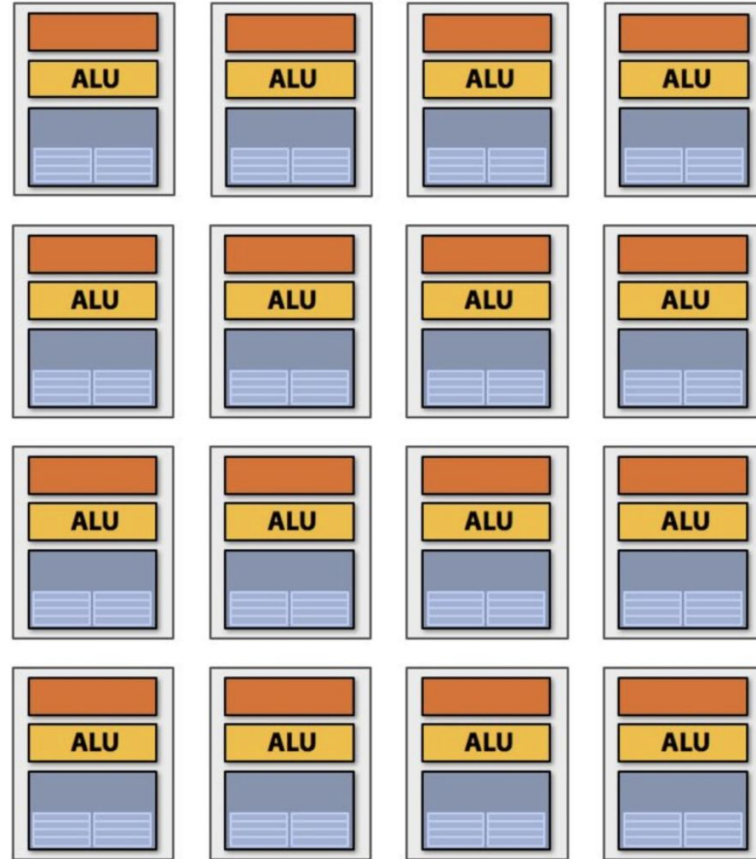
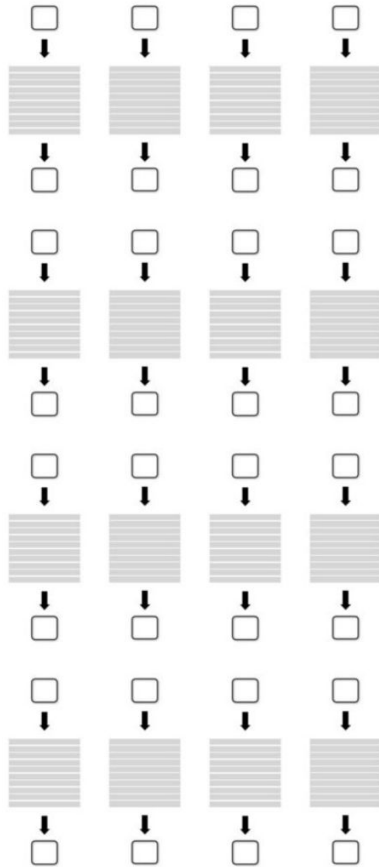
```
void thread_func(int start, int end, int M, float* x, float* y) {
    for (int i = start; i < end; i++) {
        y[i] = 0; // Initialize energy
        for (int j = 0; j < M; j++) {
            int index = i * M + j;
            float v = x[index + M];
            float m = x[index];
            y[i] += 0.5 * m * v * v;
        }
    }
}

void calculateKineticEnergyParallel(int N, int M, float* x, float* y) {
    std::thread t(thread_func, 0, N / 2, M, x, y); // First half
    thread_func(N / 2, N, M, x, y); // Second half
    t.join(); // Wait for thread to complete
}
```

4 Cores → 4 Instruction Streams



16 Cores → 16 Instruction Streams



Spawning Multiple Threads

- Parallelism is across iterations of the loop. the iterations of the loop carry out the exact same sequence of instructions (defined by the loop body)
- But on different input data given by X

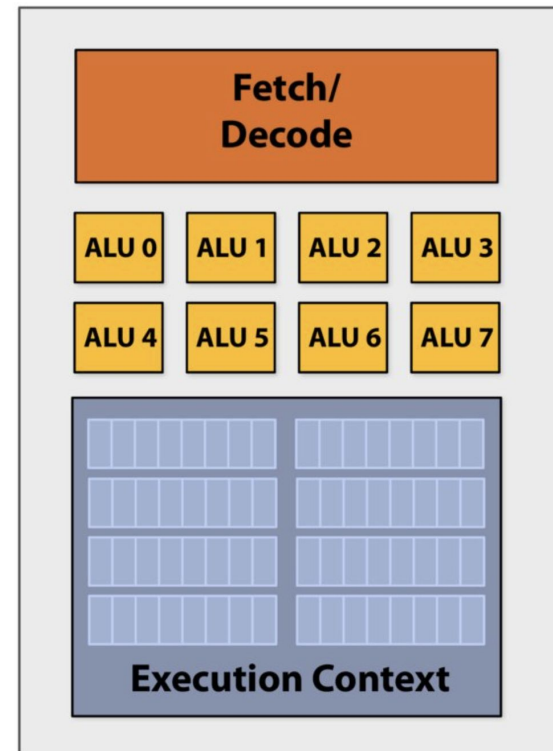
```
void thread_func(int start, int end, int M, float* x, float* y) {  
    for (int i = start; i < end; i++) {  
        y[i] = 0; // Initialize energy  
        for (int j = 0; j < M; j++) {  
            int index = i * M + j;  
            float v = x[index + M];  
            float m = x[index];  
            y[i] += 0.5 * m * v * v;  
        }  
    }  
}  
  
void calculateKineticEnergyParallel(int N, int M, float* x, float* y) {  
    std::thread t(thread_func, 0, N / 2, M, x, y); // First half  
    thread_func(N / 2, N, M, x, y); // Second half  
    t.join(); // Wait for thread to complete  
}
```


Key Concept

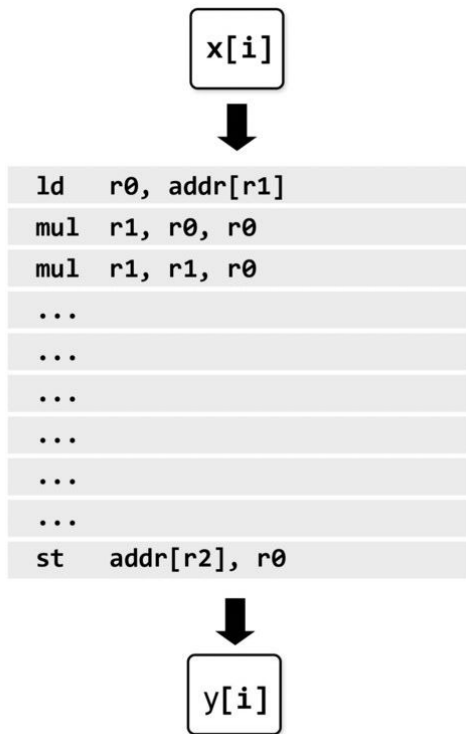
- **Parallelism Across Loop Iterations:**
 - Executes the **same instructions** for each loop iteration.
 - Operates on **different input data** (e.g., array $x[i]$).

SIMD Architecture

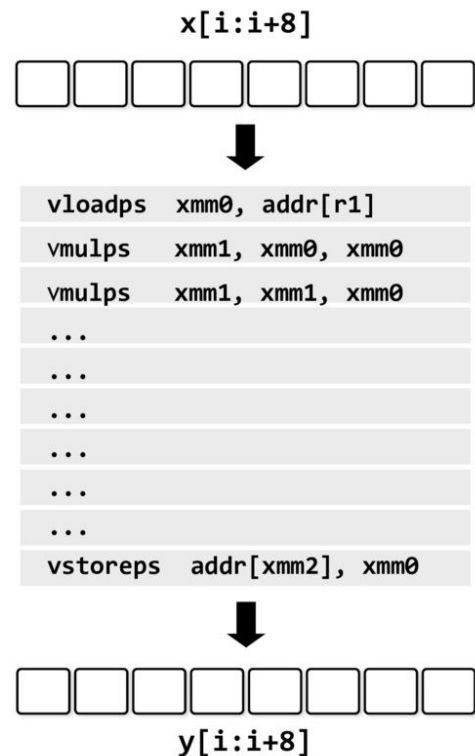
- **Fetch/Decode Unit:**
 - Fetches and decodes the instructions once for all iterations.
- **ALUs (Arithmetic Logic Units):**
 - Execute the same operation (e.g., multiplication) on multiple data points in parallel.
- **Execution Context:**
 - Stores the input data and states for all ALUs.



Sequential



Vector Program



Sequential

```
void calculateKineticEnergy(int N, int M, float* x, float* y) {
    for (int i = 0; i < N; i++) {
        y[i] = 0;
        for (int j = 0; j < M; j++) {
            int index = i * M + j;
            float v = x[index + M];
            float m = x[index];
            y[i] += 0.5f * m * v * v;
        }
    }
}
```

- **AVX Parallelism:** Processes 8 floating-point values simultaneously using 256-bit registers, significantly accelerating operations like multiplication and addition.
- **Loop Vectorization:** Replaces the inner loop with SIMD instructions to compute kinetic energy for multiple particles in parallel, reducing iterations and improving efficiency.

Vector program (using AVX intrinsics)

```
#include <immintrin.h> // For AVX intrinsics

void calculateKineticEnergyAVX(int N, int M, float* x, float* y) {
    const __m256 half = _mm256_set1_ps(0.5f); // Set 0.5f in all lanes of the A

    for (int i = 0; i < N; i++) {
        __m256 sum = _mm256_setzero_ps(); // Initialize the SIMD register to st

        for (int j = 0; j < M; j += 8) { // Process 8 elements at a time
            int index = i * M + j;

            // Load 8 masses (m) and 8 velocities (v) into AVX registers
            __m256 m = _mm256_loadu_ps(&x[index]);
            __m256 v = _mm256_loadu_ps(&x[index + M]);

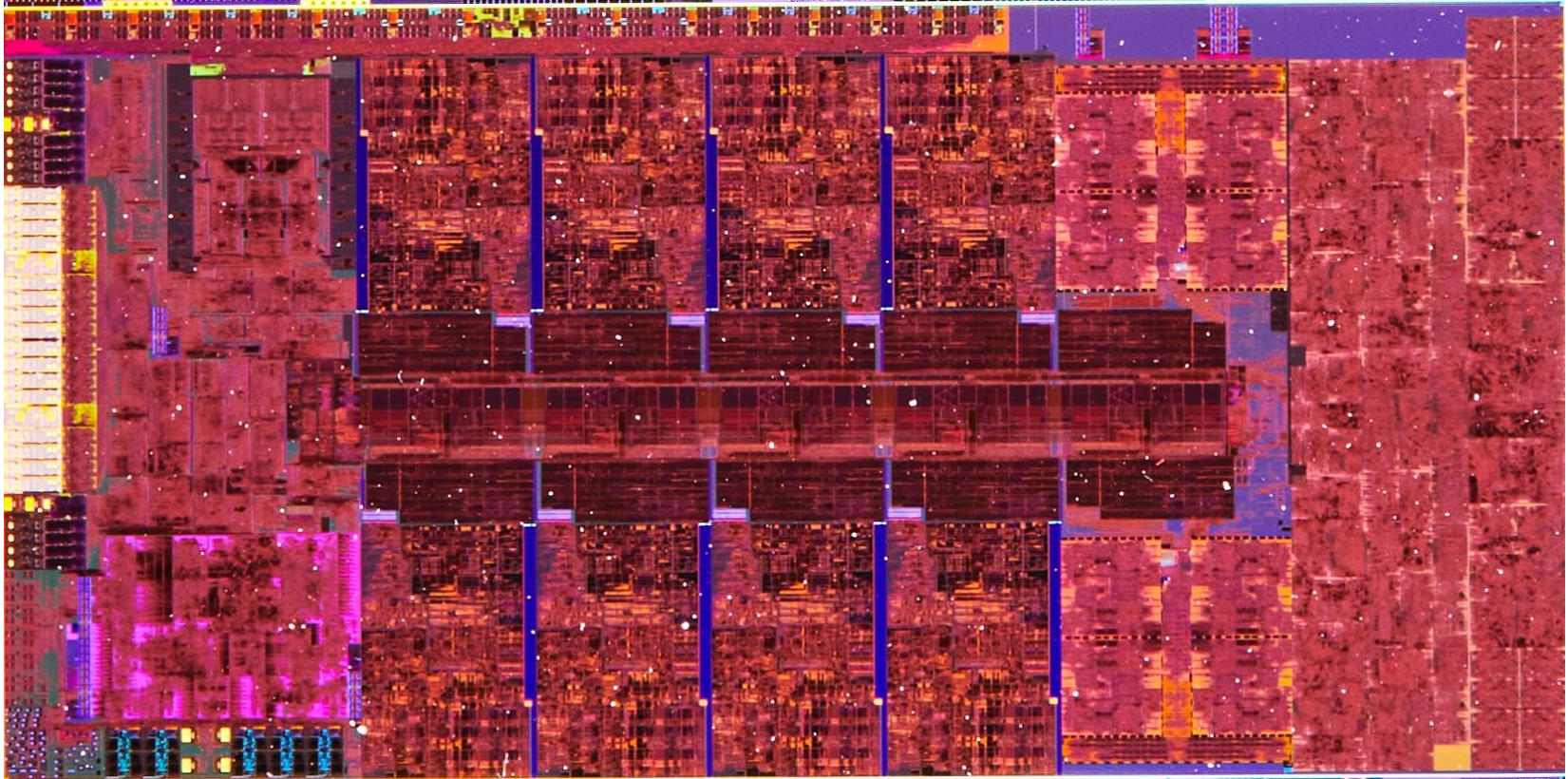
            // Compute v * v (velocity squared)
            __m256 v_squared = _mm256_mul_ps(v, v);

            // Compute 0.5 * m * v^2
            __m256 kinetic_energy = _mm256_mul_ps(half, _mm256_mul_ps(m, v_squared));

            // Accumulate the result into the sum register
            sum = _mm256_add_ps(sum, kinetic_energy);
        }

        // Horizontally add all 8 lanes in the 'sum' register and store the res
        float temp[8];
        _mm256_storeu_ps(temp, sum);
        y[i] = temp[0] + temp[1] + temp[2] + temp[3] + temp[4] + temp[5] + temp[6] + temp[7];
    }
}
```

Intel multi-core Alder Lake 2022



Source: <https://x.com/aschilling/status/1453391035577495553/photo/2>

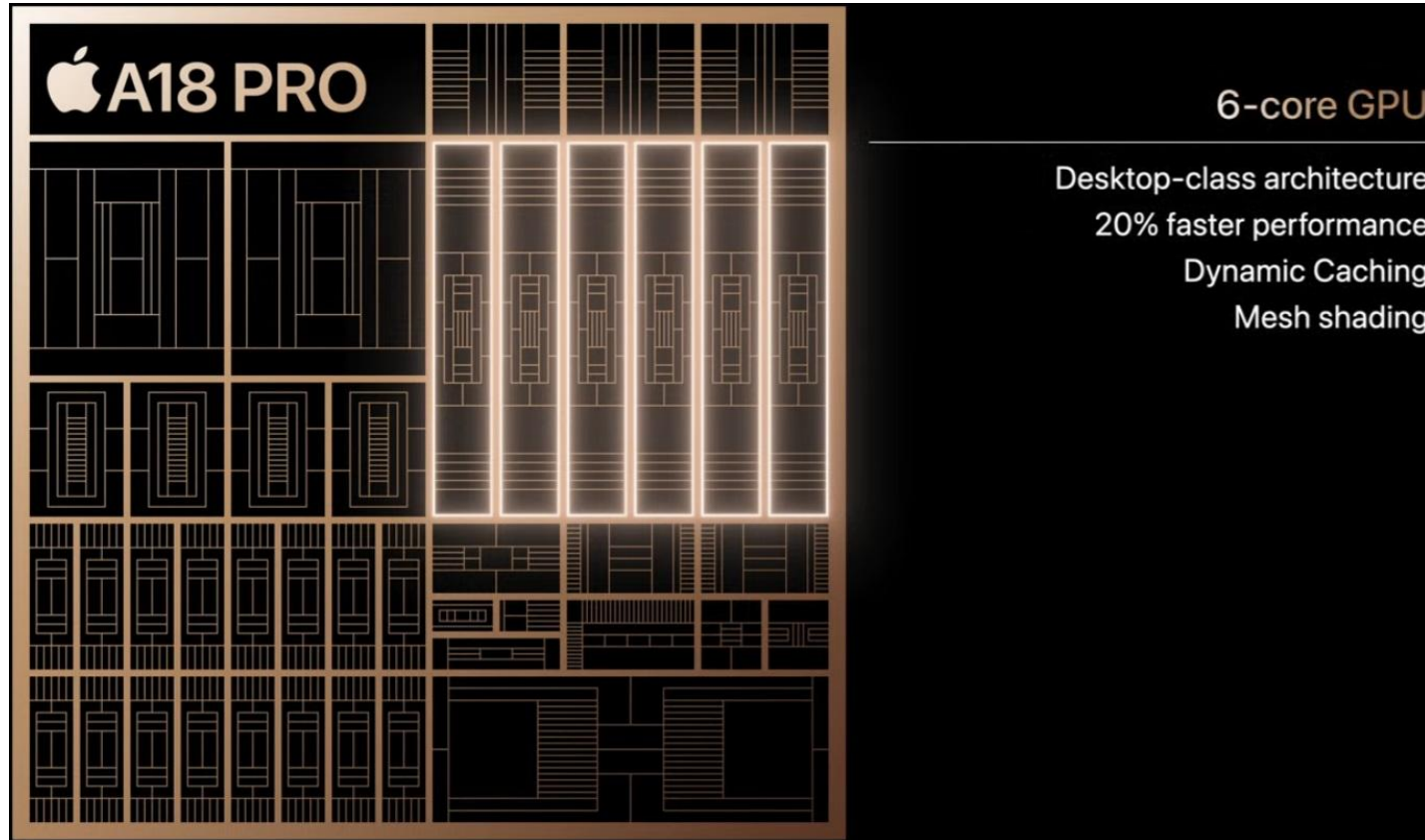
Intel multi-core Alder Lake 2022



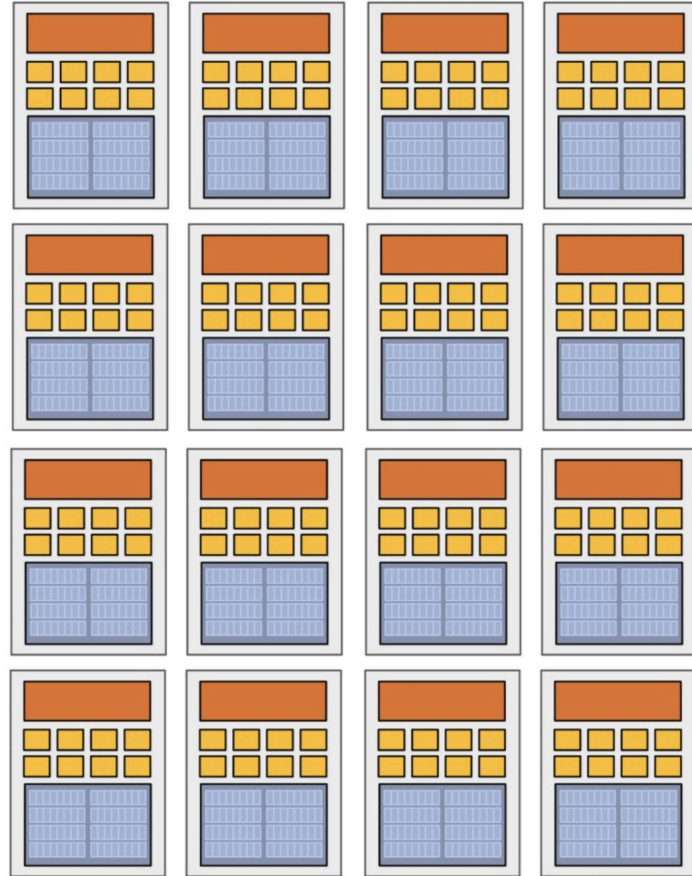
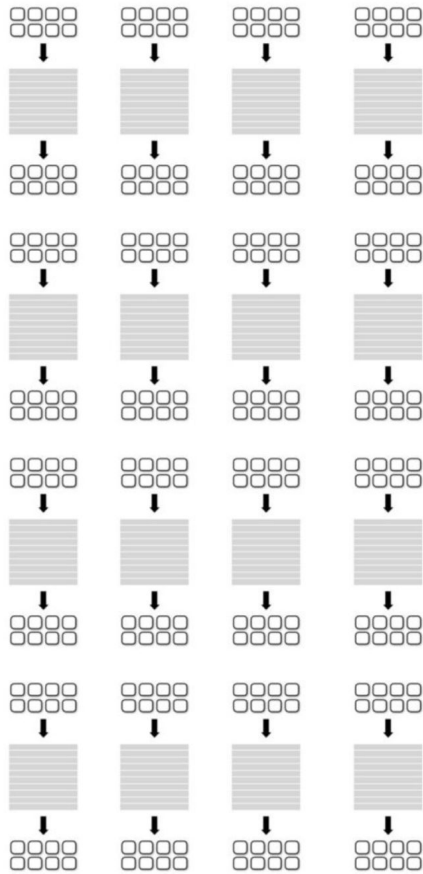
NVIDIA V100 Architecture



Apple A18 Pro



16 SIMD cores: 128 elements in parallel



SIMD Instruction Sets

- **Intel AVX2:** 256-bit operations
 - Handles 8×32 -bit floats or 4×64 -bit doubles per instruction.
- **Intel AVX-512:** 512-bit operations
 - Processes 16×32 -bit floats or 8×64 -bit doubles.
- **ARM Neon:** 128-bit operations
 - Supports 4×32 -bit floats or 2×64 -bit doubles.

Compiler-Generated SIMD

- **Programmer-Controlled Parallelism:**
 - Explicit SIMD via intrinsics (e.g., `_mm256_mul_ps`).
 - Parallel languages like forall convey intent.
- **Auto-Vectorization:**
 - Compilers analyze loops for dependencies and generate SIMD instructions automatically.

Implicit SIMD

- **Definition:** Compiler generates scalar instructions; hardware executes SIMD.
- **Control:** Managed by the **hardware** at runtime.
- **Parallelism:** Executes the same instruction for multiple program instances on different data.
- **Examples:** Used in **GPUs** (e.g., NVIDIA, AMD) with wide SIMD lanes.
- **Optimization:** Performance depends on the hardware's ability to manage **divergence**.
- **Key Advantage:** Easier for programmers but can lose efficiency due to **thread divergence**.

Explicit SIMD

- **Definition:** Programmer explicitly requests SIMD using **intrinsics or vectorized code**.
- **Control:** Managed by the **programmer** at compile time.
- **Parallelism:** SIMD registers (e.g., AVX, Neon) operate on multiple data points in parallel.
- **Examples:** Used in **CPUs** with AVX, AVX-512, or ARM Neon intrinsics.
- **Optimization:** Programmer optimizes **loop unrolling and vector operations** for performance.
- **Key Advantage:** **Higher efficiency** if optimized correctly for the architecture.

Superscalar Processing:

- Executes multiple instructions from the same instruction stream in parallel within a single core.
- The hardware dynamically identifies and exploits instruction-level parallelism (ILP) during execution.

SIMD (Single Instruction, Multiple Data):

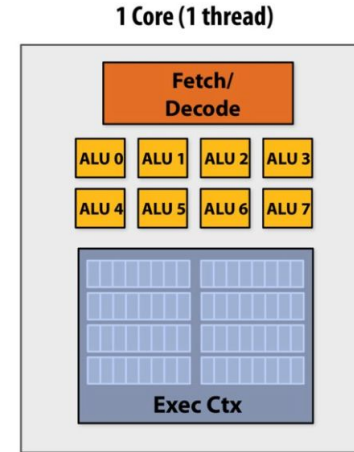
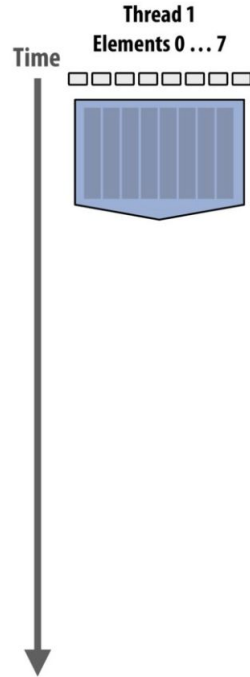
- Uses multiple ALUs that operate under the control of a single instruction, executing the same operation on multiple data points in parallel within a core.
- Well-suited for data-parallel workloads, reducing control overhead across many ALUs.
- Vectorization can be performed by the compiler (explicit SIMD) or by the hardware at runtime (implicit SIMD).

Multi-Core Processing:

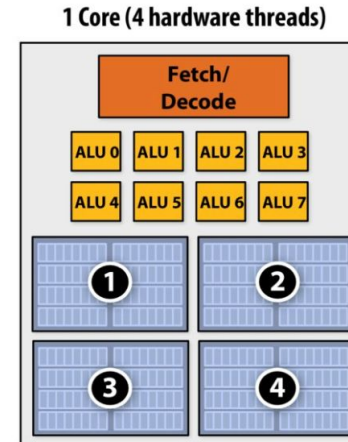
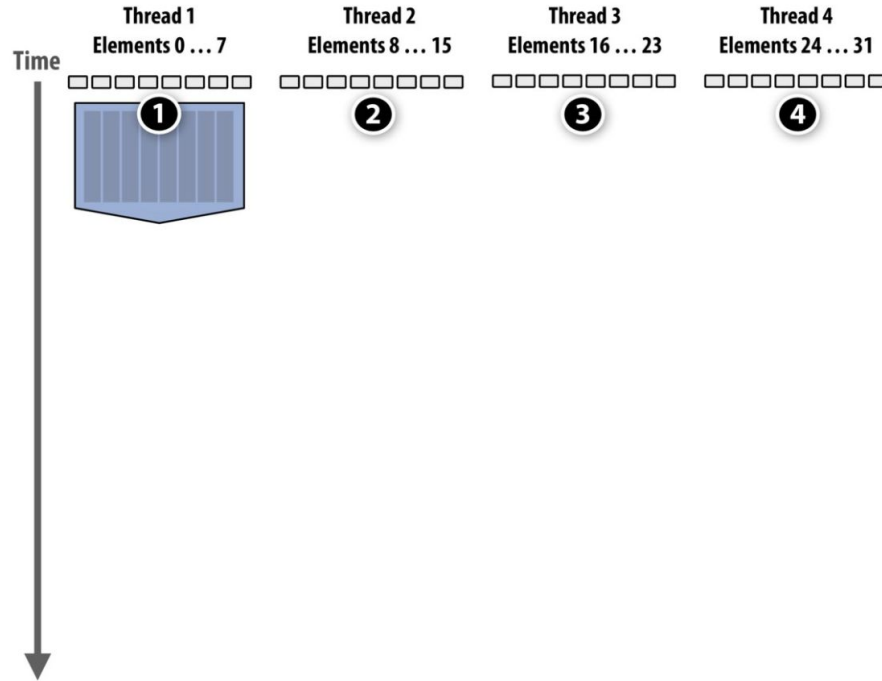
- Employs multiple processor cores to execute different instruction streams concurrently.
- Enables thread-level parallelism, where software creates and manages threads to expose parallelism to the hardware (e.g., via threading APIs).

YOU CAN COMBINE ALL THREE FOR MORE PARALLELISM

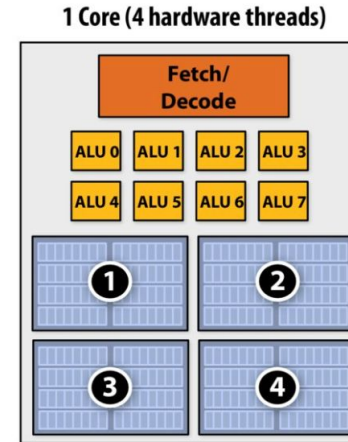
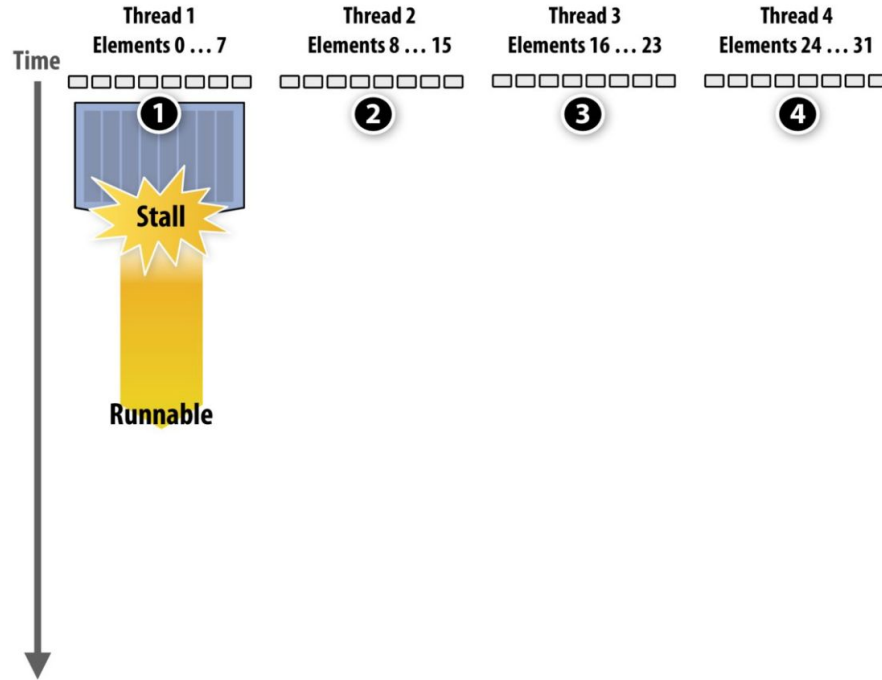
Hiding Stall With Multi-Threading



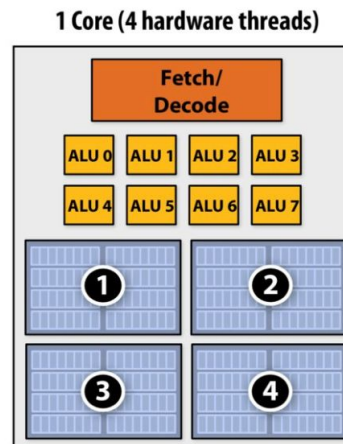
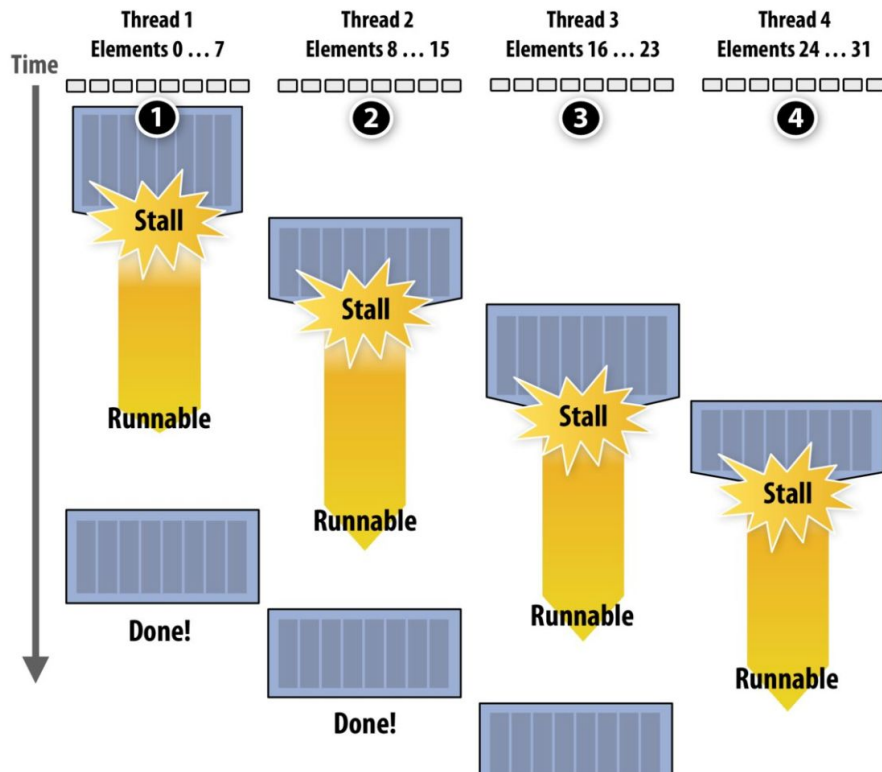
Hiding Stall With Multi-Threading



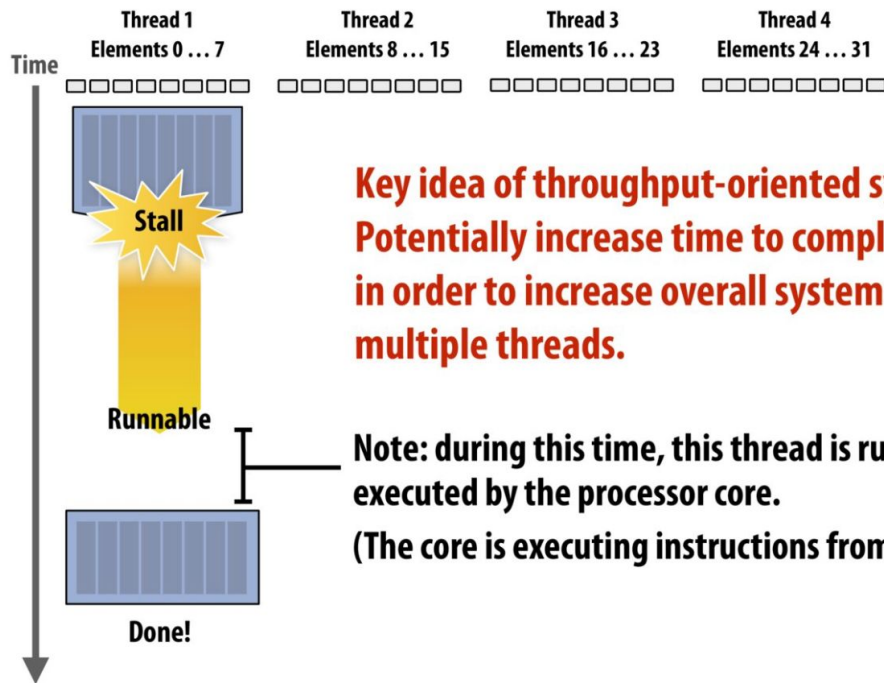
Hiding Stall With Multi-Threading



Hiding Stall With Multi-Threading



Hiding Stall With Multi-Threading



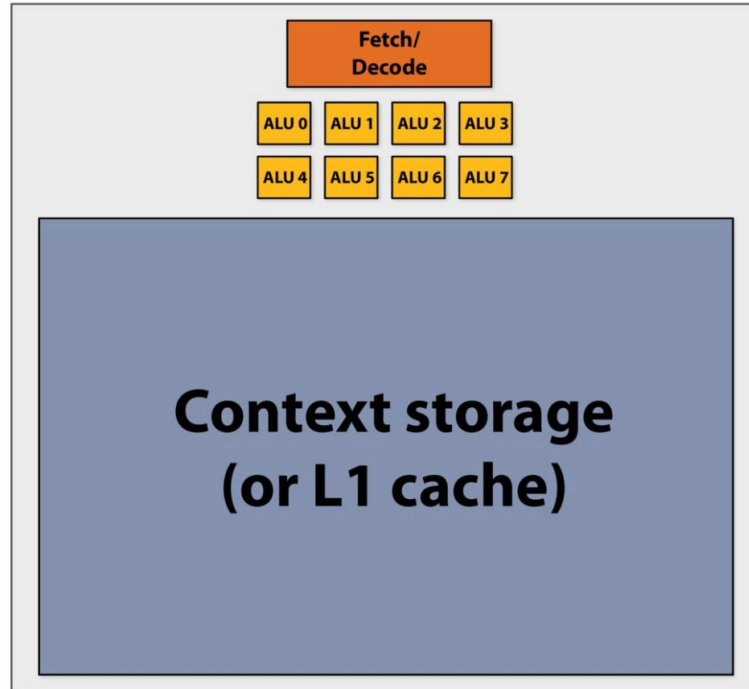
Key idea of throughput-oriented systems:

Potentially increase time to complete work by any one thread, in order to increase overall system throughput when running multiple threads.

Note: during this time, this thread is runnable, but it is not being executed by the processor core.

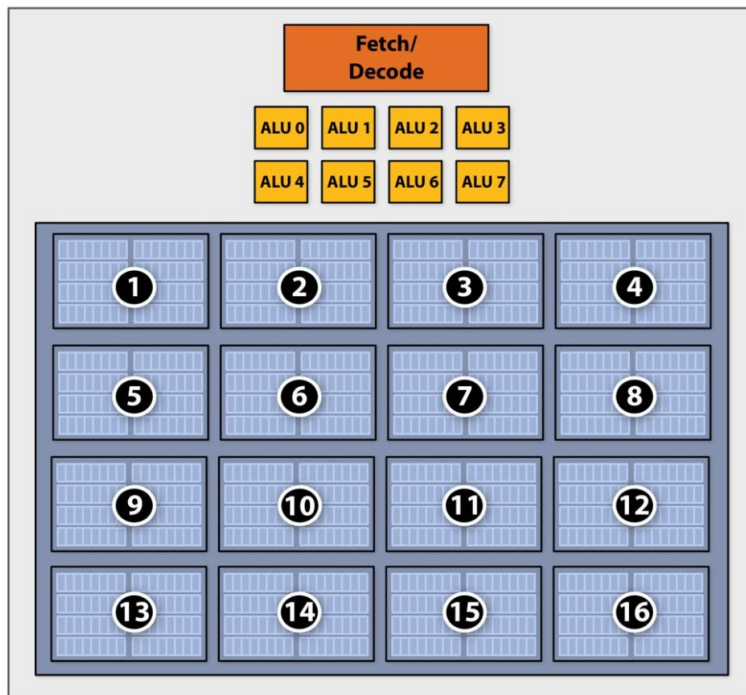
(The core is executing instructions from another thread.)

Consider on-chip storage of execution contexts as a finite resource

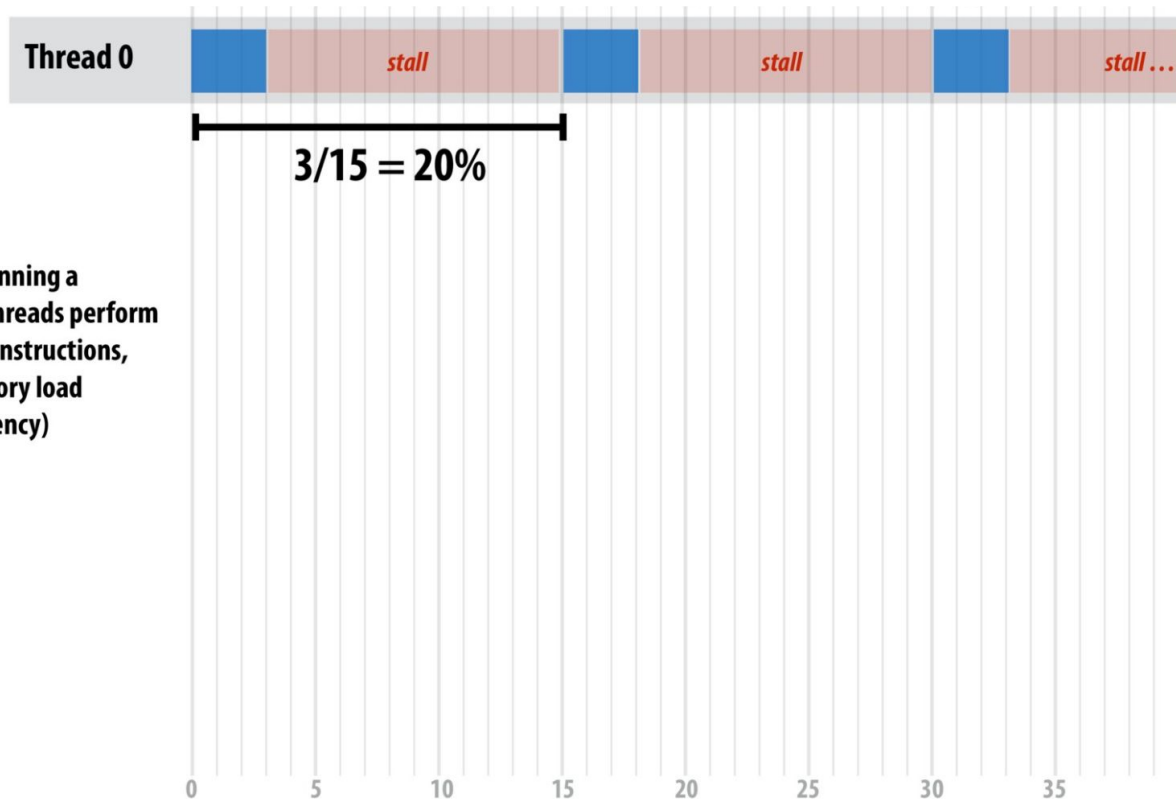


Large Context vs. Many Small Context

16 hardware threads: storage for small working set per thread

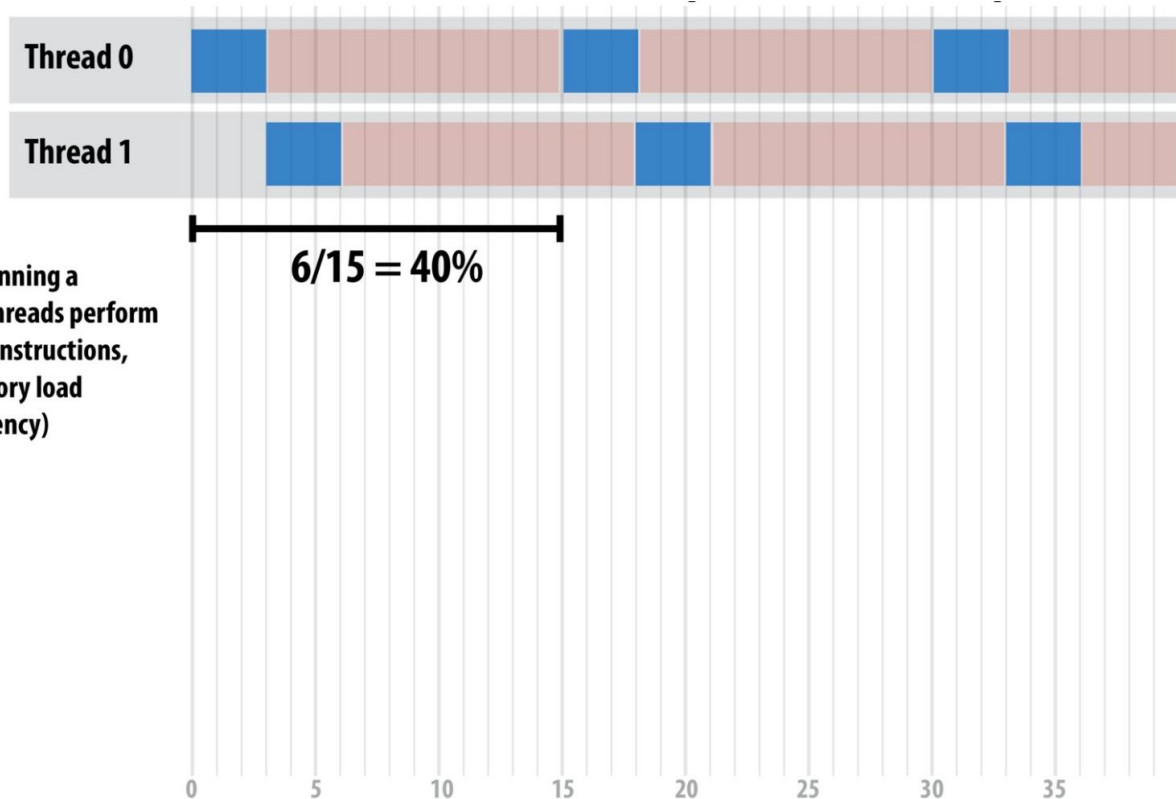


Utilizing One Core One Thread



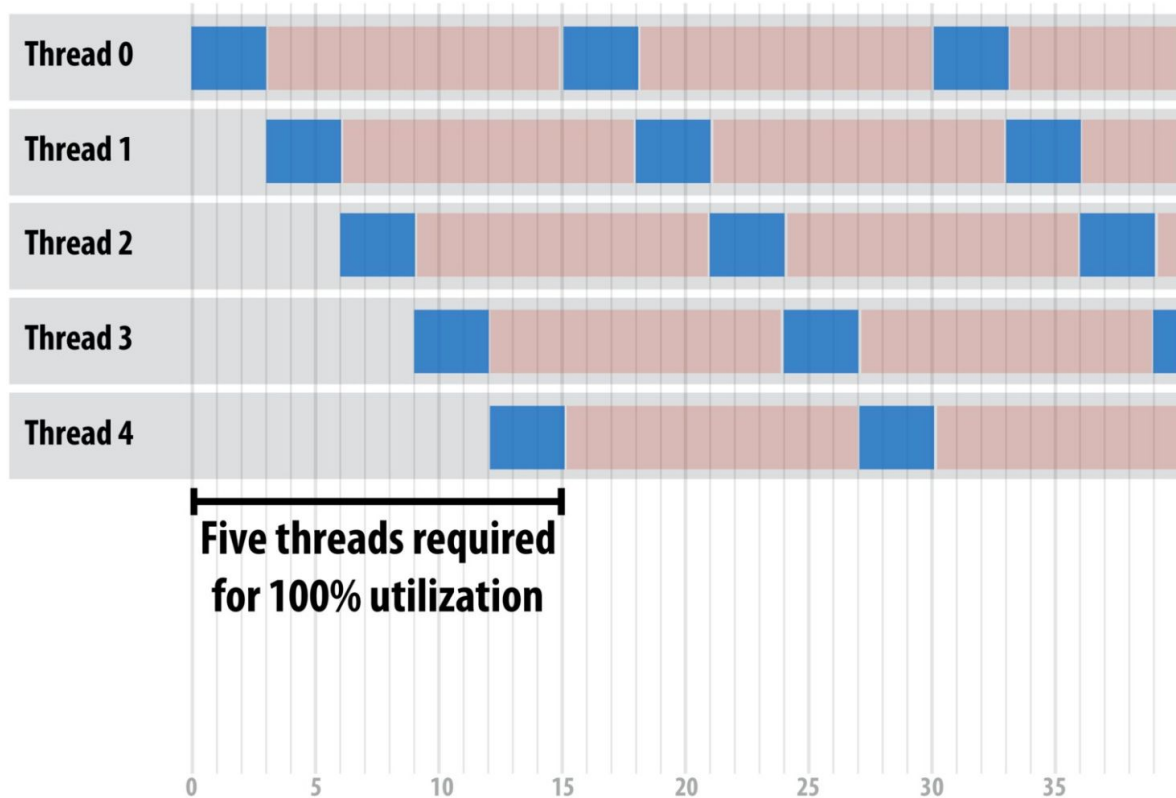
Assume we are running a program where threads perform three arithmetic instructions, followed by memory load (with 12 cycle latency)

Utilizing One Core Two Thread

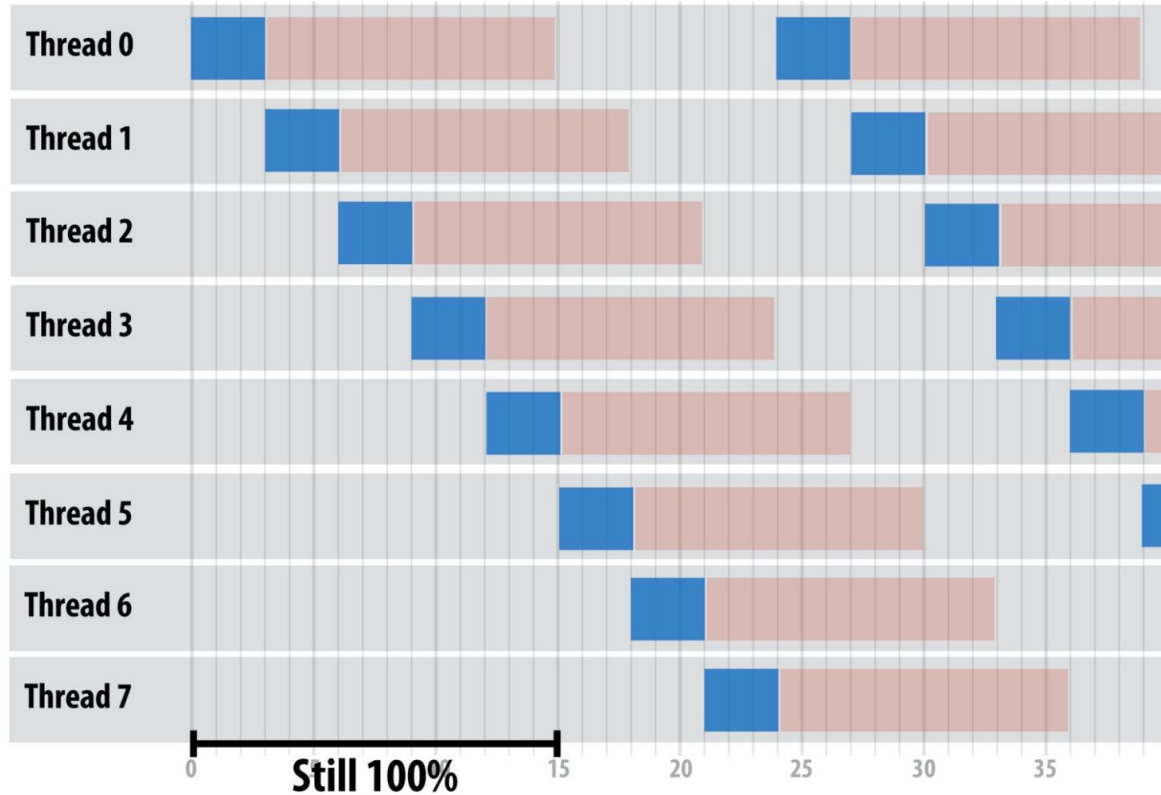


Assume we are running a program where threads perform three arithmetic instructions, followed by memory load (with 12 cycle latency)

Utilizing One Core Five Thread

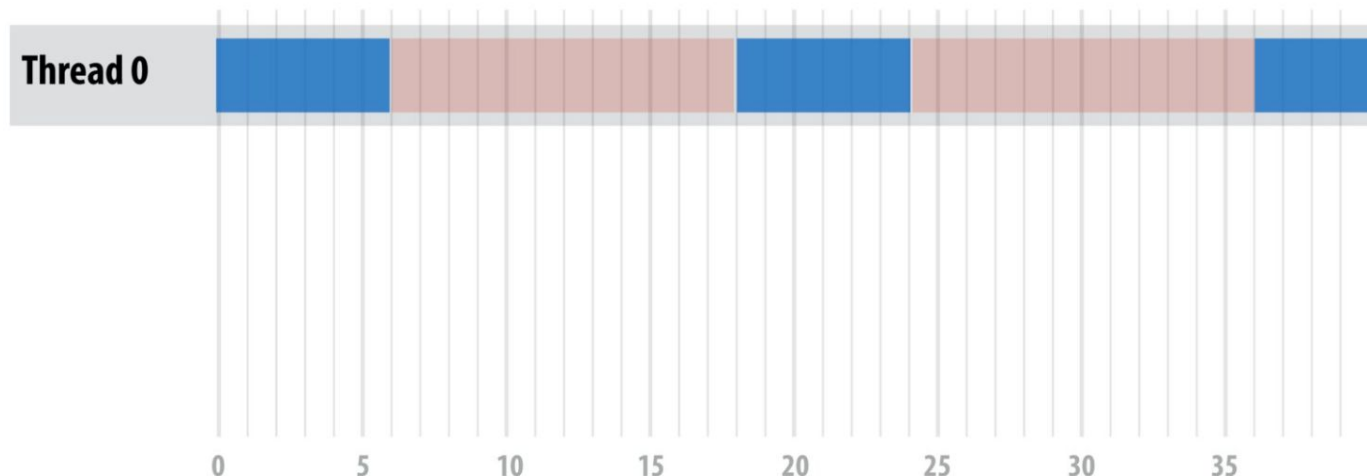


Utilizing One Core Thread > 5



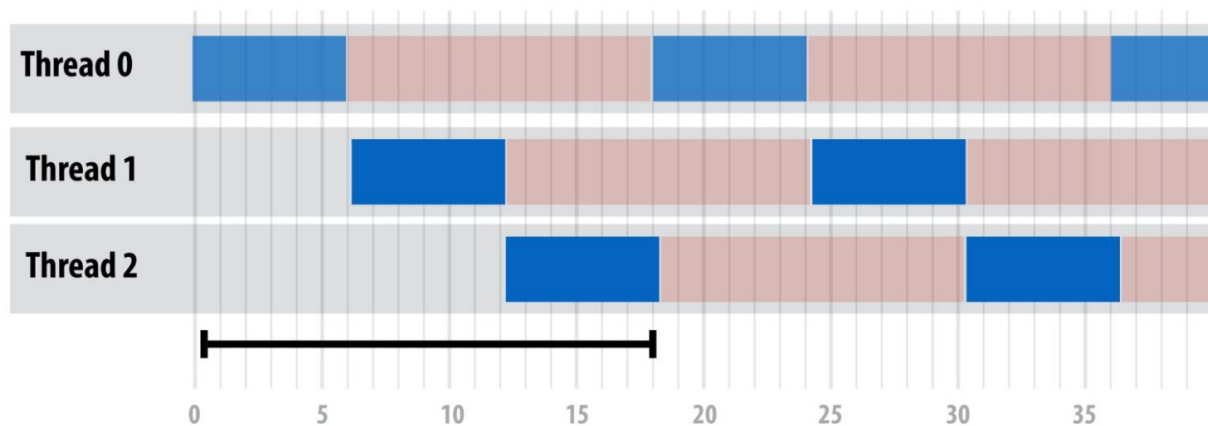
Utilization of Compute Heavy Computation

Threads now perform *six arithmetic instructions*, followed by memory load (with 12 cycle latency)



Utilization of Compute Heavy Computation

Threads now perform *six arithmetic instructions*, followed by memory load (with 12 cycle latency)



**100% utilization using
only three threads**

- ▷ A processor equipped with multiple hardware threads can **reduce idle time (stalls)**
- ▷ by executing instructions from other threads
- ▷ while one thread is waiting for a high-latency operation, such as a memory access
- ▷ Importantly, **multithreading does not decrease the latency** of the operation itself;
- ▷ **multithreading** ensures that the processor's resources remain **utilized** by switching to other ready-to-run threads.

Problem Without Multithreading:

- ▷ In single-threaded processors, when a thread encounters a **high-latency operation** (e.g., fetching data from memory), the processor sits idle, wasting valuable resources.

How Multithreading Solves This:

- ▷ By maintaining **multiple active threads**, the processor can switch to another thread and continue executing instructions while waiting for the stalled thread to resume.
- ▷ This technique improves **processor utilization** and reduces the impact of stalls.

- ▷ A multi-threaded processor reduces the impact of memory latency by performing arithmetic operations from other active threads while waiting for memory access to complete.
- ▷ Programs with a **higher ratio of compute to memory** require fewer threads to effectively hide memory stalls, as they spend more time computing and less time waiting for memory.

Memory Latency and Stalls:

- ▷ Accessing data from memory can introduce significant delays, stalling the processor.

Compute-to-Memory Ratio:

- ▷ Programs with **intensive arithmetic computations** per memory access are naturally less affected by memory latency.

Implication:

- ▷ For memory-bound programs, adding more threads is crucial to hide latency effectively.
- ▷ For compute-heavy programs, fewer threads can achieve high processor utilization.

1. Core Manages Multiple Threads

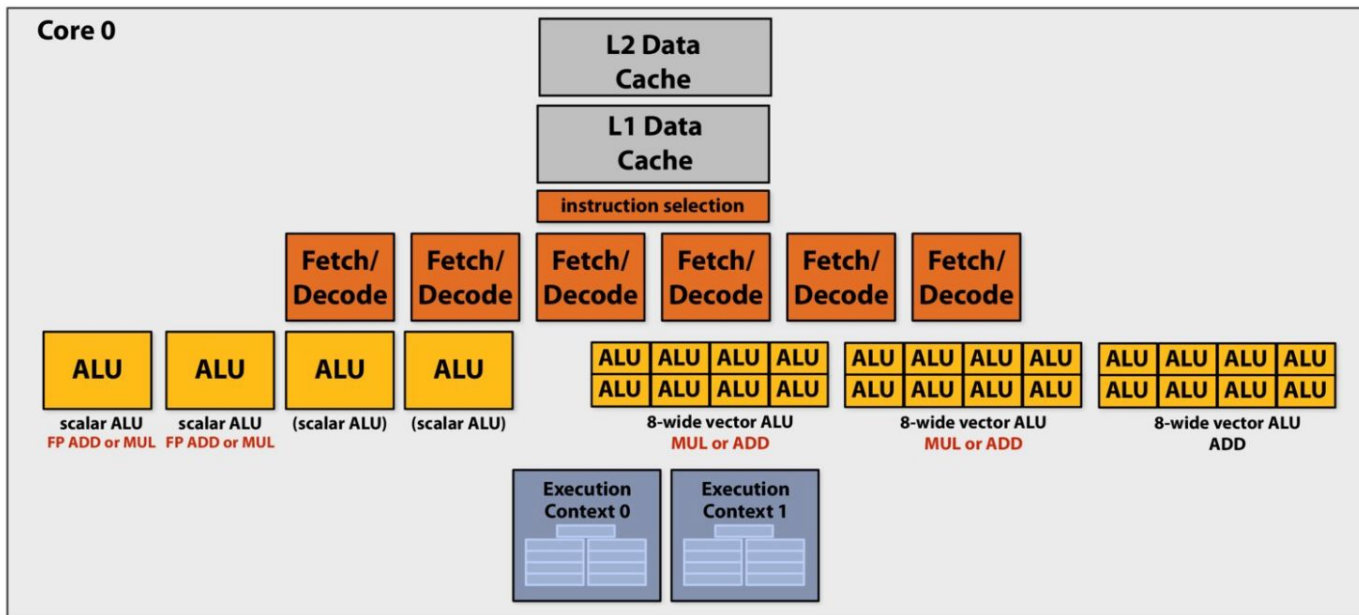
- ▷ The core still has the same **ALU resources**, but multi-threading helps utilize them more efficiently by mitigating high-latency operations like memory access.
- ▷ The **processor decides** which thread to run in each clock cycle, ensuring better resource utilization.

2. Interleaved Multithreading (Temporal Multi-Threading)

- ▷ At every clock cycle, the core **switches between threads**, selecting one to run its instructions on the ALUs.
- ▷ Example: **Rotating thread scheduling** where one thread runs while others wait.

3. Simultaneous Multithreading (SMT)

- ▷ At each clock cycle, the core executes instructions from **multiple threads simultaneously** on available ALUs.
- ▷ Example: **Intel Hyper-Threading** enables **2 threads per core** to share resources and execute concurrently.



Two-way multi-threaded cores (2 threads).
Each core can run up to four independent scalar instructions
and up to three 8-wide vector instructions
(up to 2 vector mul or 3 vector add)

GW

The diagram illustrates the architecture of a GPU with 64 warps. The top row shows four 'Warp Selector' blocks, each with a 'Fetch/Decode' unit. Below these are four grids of colored squares representing the execution of 64 warps. The bottom row shows four 'Warp' blocks, each containing a grid of 64 warps (Warp 0, Warp 4, ..., Warp 60, Warp 64). A large blue box at the bottom indicates 'Shared memory + L1 cache storage (128 KB)'.

**64 x 32 = up to 2048 data items
processed concurrently per "SM" core**

Registers divided among
(up to) 64 "warps" per SM

 = Tensor core unit
 = Load/store unit

- ▷ "Introduction to Parallel Computing" by Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar
- ▷ "Computer Architecture: A Quantitative Approach" by John L. Hennessy and David A. Patterson
- ▷ Stanford CS149
 - Highly recommended:
<https://www.youtube.com/watch?v=V1tINV2-9p4&list=PLoROMvodv4rMp7MTFr4hQsDEcX7Bx6Odp>